# Pre-Screening Assessment Answers - Front-End Engineering

## Position Applied For: Front-End Engineer

## Part A - Questions

### Section 1 - Code

### Answer to Q1

Factually Wrong Statements:

1. React.memo performs a shallow comparison of props by default, not a deep comparison.
2. React.memo is not broken; it re-renders intentionally because its props fail shallow reference equality checks.

Proposed Fix & Conditions: Wrapping columns in useMemo preserves its reference across renders, but it will not fix the issue if:

- ProductRow receives other unstable props.
- ProductRow directly consumes a React Context that updates on every keystroke via useContext.

Two Other Defeaters:

1. Unmemoized inline callback functions passed as props (e.g., onSelect={() => handleSelect(id)}) without useCallback.
2. Inline object/array literals or inline style objects passed as props (e.g., style={{ color: 'red' }}).

Actual Cause: The actual cause of the re-render is the parent component updating its state on every search input keystroke, whereas unstable prop references are what cause that parent re-render to expensively propagate down to every memoized row component.

## Answer to Q2

### 1. Defects

- Defect 1: Missing Optimistic Rollback (Severity: 1)
    - What the user sees: The user changes a product status, and the UI updates instantly. However, if the server request fails, the UI incorrectly retains the new status instead of reverting, silently deceiving the user.
    - Exact conditions: The mutation API call fails (e.g., network timeout, 500 error) while the user is viewing an unfiltered product list.
    - Why it is first: Silent data corruption is inherently more severe than degraded UI performance. The user falsely assumes a business action succeeded when it actually failed.
- Defect 2: Hardcoded Cache Key for Optimistic Update (Severity: 2)
    - What the user sees: The user experiences a delayed status update (waiting for the network roundtrip) rather than an instant, optimistic UI change.
    - Exact conditions: The user applies any filter to the list (e.g., ?category=tech), then updates a status. The updateQueryData targets the exact cache key {} as ProductFilters, so it completely misses the cached data for the filtered query.

### 2. False Claim

The comment // ignore - the invalidation will refetch anyway is factually false. In RTK Query, invalidatesTags only triggers automatically upon a successful mutation. Because a failed request throws an error and lands in the catch block, the invalidation never fires. Consequently, the refetch does not happen, leaving the falsely updated optimistic data trapped in the client cache.

### 3. Looks Like a Defect, But Isn't

The statement row.status = status appears to be a defect because directly mutating state is a fundamental Redux anti-pattern. However, it is perfectly fine here. RTK Query's updateQueryData utilizes Immer under the hood. Immer wraps the provided draft state in a proxy, safely translating direct, mutable assignments into immutable updates.

## Answer to Q3

```
export const SupplierBadge = memo(function SupplierBadge({ supplierId }: { supplierId: string }) {

 const { data, isLoading, isError } = useGetSupplierQuery(supplierId);



 if (isLoading) return &lt;Skeleton className="h-5 w-24" /&gt;;

 if (isError) return null;



 return (

  &lt;span className="rounded bg-muted px-2 py-0.5 text-xs"&gt;

   {data?.name?.toUpperCase() || &apos;&apos;}

  &lt;/span&gt;

 );

});
```

### Removals & Problems Caused

- useState & useEffect: Caused an unnecessary, expensive double-render cycle when data arrived (render with stale state $\rightarrow$ effect runs $\rightarrow$ state updates $\rightarrow$ render with fresh state).
- useMemo: Imposed hook-execution overhead that exceeded the trivial CPU cost of running .toUpperCase() directly during render.
- useSupplierName: Unnecessary custom hook boilerplate that only obfuscated the component.

### The User-Visible Bug

- Removal fixing it: Removing useState and useEffect.

- Sequence of events:
    1. A row renders with supplierId="1". The name "ACME" loads, and the effect sets the state.
    2. The table re-sorts or paginates, passing this exact component instance a new supplierId="2".
    3. The query fetches supplier 2. If supplier 2's name is missing, null, or an empty string, the condition if (data?.name) fails.
    4. setName is skipped, leaving the badge permanently displaying the stale "ACME" data for the new row.

### The 200-Row Problem

- Problem: The N+1 query problem. The table makes 200 individual network requests to fetch supplier names instead of one.
- Layer: The Data/Backend layer (the backend should join the supplier name into the original table payload, or the API should expose a batched endpoint).

## Answer to Q4

### 1. Painted Values

idle -> saving -> idle.

### 2. Unpainted Values

The value 'saved' is assigned but never painted. In React 18, state updates following an await are automatically batched. The final setLabel executes in the exact same event loop tick as setLabel('saved'), instantly overwriting it before React yields to the browser for a render and paint.

### 3. Final Value & Lost Feedback

The final displayed value is idle. Because of stale closure, the variable label inside the onClick function evaluates to its value from the initial render ('idle'). Thus, the ternary evaluates to 'idle', overwriting the success state. The user has completely lost the success feedback, falsely believing the action did nothing or was reverted.

### 4. Smallest Fix

Remove the final `setLabel(label === 'saving' ? 'done' : label);` line entirely. The `try/catch` blocks already correctly and exhaustively handle the terminal states.

## Section 2 - Design and judgement

## Answer to Q5

```typescript
type ApiResult<T> =
  | { ok: true; data: T; meta: { page: number; pageSize: number; total: number } }
  | { ok: false; error: { code: ErrorCode | (string & {}); message: string; field?: string } };
```

```typescript
interface RequestOptions {
  // 'toast' for tables, 'silent' for polling, 'manual' for forms
  errorHandling?: 'toast' | 'silent' | 'manual';
}
```

```typescript
declare function apiClient<T>(endpoint: string, options?: RequestOptions): Promise<ApiResult<T>>;
```

## Answers

1. Non-null data without casting: ApiResponse<T> is a discriminated union. When you check if (response.error === null), TypeScript's control flow analysis automatically narrows the type to the success branch. In that branch, data is explicitly typed as T, guaranteeing it is not null without requiring type assertions.
2. Compile error on new members: By using an exhaustiveness check. If you evaluate error.code in a switch statement, the default case must assign error.code to a variable explicitly typed as never. If a new code is added to the union but missing from the switch cases, TypeScript will throw a compile error because the new string literal cannot be assigned to never.
3. Handling unknown runtime codes: The default switch case from the exhaustiveness check handles this. While it forces a compile error for unhandled known types, at runtime it must execute fallback logic—such as defaulting to a generic error toast or surfacing the backend's raw message string—so the error remains visible to the user rather than being silently ignored.
4. Mismatched fields: If error.field doesn't match, the form state library (e.g., React Hook Form) will attempt to attach the error to an unregistered input, swallowing the message and leaving the user confused. This is fixed in the specific Form component (or a dedicated form-adapter hook), which must map backend field keys to frontend input names using a translation object before calling the form's error-setting function.

## Answer to Q6

## Reject the proposal.

What breaks and for whom: Virtualization intentionally removes off-screen elements from the Document Object Model (DOM) to save memory. Consequently, both native browser search (Ctrl-F) and native printing (Ctrl-P) will completely break for the

warehouse team. Ctrl-F will fail to locate any order number not currently visible in the scroll viewport, and Ctrl-P will only print the immediate screen (or blank space) rather than the entire filtered list.

Alternative approach: Implement standard pagination (e.g., 100 rows per page) to ensure the table becomes instantly interactive. To support the user's workflow, introduce a dedicated application-level search input to filter the dataset in memory, replacing Ctrl-F. Finally, implement a dedicated "Print Full List" button. This button should temporarily render a visually hidden, drastically simplified table containing the entire filtered dataset, styled specifically for @media print, before invoking window.print().

Cost: The primary cost is user retraining and workflow friction; the warehouse team must break their ingrained habit of using native Ctrl-F and rely on the custom search input instead. The secondary cost is the engineering overhead required to build, test, and maintain the separate print-optimized view structure.

## Answer to Q7

I choose (b).

Justification: Silent data loss is a critical, destructive failure that destroys user trust and actively prevents users from successfully completing tasks. A 4-second UI freeze is a frustrating performance degradation, but the table is still fundamentally functional once loaded. Guaranteeing data integrity always supersedes performance optimization.

What stays broken: The products table will continue to freeze the browser tab for roughly 4 seconds every time the 12,000 rows are loaded.

What I say to the stakeholder for (a): "While the 4-second table freeze is a undeniably poor experience, silently deleting user input makes our forms completely unusable. We must prioritize unblocking core workflows and ensuring data integrity for this demo; we will schedule the table virtualization immediately afterward."

## Answer to Q8

Pair 1: Requirements 2 & 3

- Conflict: The frontend cannot display the exact list of affected SKUs (Req 3) if those products reside on unloaded pages and their IDs have not actually been fetched from the server yet (Req 2).
- Keep: Requirement 2.
- Ticket: "Replace exact SKU list in confirmation dialog with a summary count of affected items."
- Question: "Will users actually read thousands of individual SKUs in a popup, or is a total count sufficient?"

Pair 2: Requirements 5 & 6

- Conflict: A strictly atomic "all-or-nothing" operation (Req 5) can only result in 100% success or 100% failure. It is impossible to have a mix of successes and failures to report in the toast (Req 6).
- Keep: Requirement 6.
- Ticket: "Drop atomicity requirement to allow for partial fulfillment."
- Question: "If one product fails validation, should the system still update the remaining valid products?"

Pair 3: Requirements 4 & 5

- Conflict: A user selecting more than 500 items forces the frontend to send multiple sequential or parallel API requests (Req 4). The frontend cannot guarantee true transaction atomicity (Req 5) across multiple independent network requests (e.g., request 1 succeeds, request 2 fails).
- Keep: Requirement 4.
- Ticket: "Chunk large payloads into batches of 500; note that cross-batch atomicity is unsupported."
- Question: "Should we enforce a hard maximum selection limit of 500 in the UI, or can the backend build a bulk-update endpoint that accepts filter parameters instead of explicit IDs?"

Final Answer Four of the original requirements (1, 2, 4, and 6) survive unchanged.

## Section 3 - Review and judgement under pressure

### Answer to Q9

## Verdict

Request changes.

## Comments

1. src/features/products/FilterBar.tsx: The clear function uses window.location.href, which forces a full page reload, directly violating AC-2. Action: Remove it and instead call onChange({ ...value, suppliers: [] }) to update the parent filter state reactively.
2. src/features/products/FilterBar.tsx: Clicking "Add" pushes the draft to the array but leaves the previously typed text in the input box. Action: Add setDraft('') inside the Add button's onClick handler so the user can immediately type the next supplier.
3. src/features/products/useProducts.ts: Adding refetchOnMountOrArgChange: true globally alters standard RTK Query cache behavior and is unrelated to the filter UI requirements. Action: Revert this change to prevent performance regressions, unless bypassing the cache is explicitly required by this ticket.
4. src/lib/date.ts: Extracting unrelated date utilities is scope creep that increases regression risk in areas untouched by this feature. Action: Revert this file and submit the shared util extraction in a separate, dedicated tech-debt PR.
5. src/features/products/FilterBar.tsx: The local suppliers array does not deduplicate inputs, meaning a user can accidentally add the exact same supplier string multiple times. Action: Update the Add button's handler to only append draft if it does not already exist in the suppliers array.

## Deliberately Ignored

I deliberately chose not to comment on the UI lacking a way to remove an *individual* supplier from the list; while it is a poor user experience, it does not technically violate the strict wording of AC-1.

## Acceptance Criteria Status

- AC-1: Cannot tell from the diff alone. While the UI component now passes a suppliers array to onChange, I would need to check the parent component and the API endpoint to verify they have been updated to accept and process an array instead of the previous supplier string.
- AC-2: Not met. The window.location.href = '/products' command explicitly triggers a full browser reload.

## Answer to Q10

### 1. Actions in the First 60 Minutes

1. Freeze the repository: Immediately restrict push access to the development branch to prevent further accidental overwrites.
2. Locate the lost SHAs: Ask the team to check their local git reflog for the pre-rewind HEAD of development and the lost PR branches. Alternatively, retrieve the lost SHAs via the Git hosting platform's activity logs or API.
3. Restore: Recreate the lost PR branches locally from those SHAs. Reset local development to the correct recovered SHA.
4. Push: Force-push the corrected branches back to the remote and unfreeze the repository.

### 2. The Blocked Developers

When: Immediately (e.g., 09:05). What: "Please pause all fetching and pushing. A force-push accidentally rewound the remote, and I am actively restoring the lost commits. Your local work is completely safe. I will ping you the moment the remote is restored."

### 3. The Business Owner

When & What: I would not tell them immediately. Because production is unaffected and data is highly recoverable, this is a standard engineering tooling hiccup. I would only mention it during the next scheduled sync if the one-hour delay materially threatened a sprint deliverable.

### 4. Permanent Changes

What: Enable strict branch protection rules on the development branch to explicitly disable force-pushes and branch deletions. Who: The Engineering Lead or Engineering Manager, with buy-in from the development team.

## Answer to Q11

1. To the Developer

Hey [Name], your work is excellent and I deeply appreciate your drive to deliver. However, we need to adjust our approach to pull requests.

Pushing 900+ lines without tests might feel faster for you, but it creates a major bottleneck for the team. It slows down reviews and drastically increases our risk of shipping bugs, as we saw last week under pressure. We must balance your coding speed with the team's ability to review safely.

Going forward, please break your PRs into smaller, logical chunks and include the necessary tests. I know it feels like overhead, but it actually speeds up our overall delivery pipeline. Let's chat tomorrow to figure out how to streamline this.

2. To the Business Owner

Hi [Name], I hear you want features shipped faster, and that is my priority too.

To achieve it sustainably, I am adjusting our internal process. Currently, work is sometimes submitted in chunks too massive to quickly verify for quality. This actually slows down our overall delivery pipeline and risks shipping bugs to our users.

I am actively working with the team to break tasks into smaller pieces, which will safely and consistently accelerate our release cadence.

# Section 4 - Experience

## Answer to Q12

I introduced a stale closure bug in a real-time dashboard component. Mechanically, I wrote a useEffect that subscribed to a WebSocket, but I omitted the selectedProjectId from the dependency array to avoid tearing down the network connection on every selection change. Consequently, the callback permanently captured the initial render's state. When live updates arrived, the handler evaluated against that stale, initial project ID and silently dropped the incoming data because it didn't match the newly active view.

The bug was live in production for about four days. It was discovered by an account manager who noticed that real-time metrics stopped updating after they switched between client profiles.

## Answer to Q13

### Delivery Dispatch Dashboard

- What & Who: An order management dashboard used by operations dispatchers for a subscriber-based student food delivery service.
- API & Contract: Built by the internal backend team. We agreed on the REST contract by defining the payload structures for the automated bot system before writing any UI code.
- Hardest thing: Synchronizing real-time subscriber delivery statuses between incoming automated webhooks and the frontend state without causing race conditions or UI flickering.

### Workflow Monitoring Interface

- What & Who: A monitoring screen for internal operations to track n8n automation pipelines and database synchronization.
- API & Contract: Built by an external backend developer. We established the API contract by sharing Supabase table schemas and iterating on standard JSON responses via a shared Postman workspace.
- Hardest thing: Normalizing highly nested, unpredictable JSON payloads from the workflow automations into a strict, type-safe frontend data grid structure.